

CPF 프레임워크 개발자 가이드

업무 코드 작성 시 바로 찾는 실무 개발 매뉴얼

Core Platform Framework

기준: 최신 로컬 Source / 2026-08-16

전체 목차

장	찾을 수 있는 내용
1. 업무 개발 기능 지도	Controller·Service·DB·TX·호출·Cache 등 자주 쓰는 CPF 기능을 목적별로 확인
2. 실행·테스트·검증 명령	전체/단독 Runtime 실행, 특정 모듈 Test, 전체 Test와 검증 명령
3. 업무 API 개발 표준	Controller → Service → Repository/DAO 기본 개발 흐름
4. Transaction	@CpfTx 기본 사용, 옵션 적용 위치, Local/Remote 경계
5. Controller	@CpfController와 Base helper, Context/Header, 응답·검증
6. Service	@CpfService와 Domain Base, 업무 로직·검증·호출 경계
7. DB·Repository/DAO	CRUD·검색·Paging·Bulk·Lock과 Persistence 선택
8. 내부 Domain 호출	Generated Typed Client, execute(), CpfResult 처리
9. 외부 연계	@CpfClient·Timeout·Retry와 고급 ServiceCall/UNKNOWN 처리
10. Cache·Lock	get/put/evict/getOrLoad와 TTL·fail-open 선택
11. Messaging	publish/Listener, 멍등성과 실패 처리
12. Logging·Audit·Security	Context, Logging/Audit, Permission/Approval 적용
13. 공통 기준정보·메시지·영업일	Code·Parameter·Message·Calendar 공통 Service
14. Test·오류 확인	Controller/Service/Repository Test와 빠른 검증
15. Starter 선택·Reference	기능 추가 시 Public Profile/Starter/Provider 선택
16. CPF 개발 기능 TOP 100	업무 개발에서 자주 찾는 Public API·메소드·명령 Quick Reference

1. 업무 개발 기능 지도

전체 목차로

업무 API 를 만들 때 반복해서 사용하는 CPF Annotation, 호출 API, 공통 메소드와 기준정보 기능을 기능군별로 정리했습니다. 필요한 Starter 와 적용 위치를 먼저 확인하면 상세 예제로 바로 연결할 수 있습니다.

업무 개발 핵심 CPF 기능 지도

기본 셋업 이후 업무 코드를 작성할 때 가장 먼저 찾는 Annotation·API·메소드를 기능군별로 묶었습니다.

업무 계층

@CpfController · @CpfService
@CpfRepository / @CpfDao
@CpfTx

업무 호출

service.method()
domainClient.execute()
Typed External Client
serviceCaller.invoke() - 고급

공통 실행

CpfContexts.transactionId()
requireText() / requireValue()
@CpfLogging · @CpfPerformance
@CpfIdempotent

공통 기준정보

codeService.required()
messageService.resolve()
parameters.requiredValue()
calendar.nextBusinessDay()

데이터 · 비동기

cacheAside.getOrLoad()
cache.get/put/evict()
messagePublisher.publish()
@CpfMessageListener

보안 · 감사

@CpfPermission
@CpfApprovalRequired
@CpfAudit
executionFacts()

Starter는 기능별 장에서 함께 표시하고, 내부 구현보다 Public API / 업무 표준 패턴을 우선합니다.

1.1 핵심 기능 Quick Summary

기능 그룹	기능 / API	용도	필요 Starter	상세
업무 계층	@CpfController / @CpfService	Web 경계와 업무 Service 표준	web-api / cpf-starter	\$5~6
DB	@CpfRepository / @CpfDao	Repository/DAO 표준	DB Starter	\$7
Transaction	@CpfTx	Local DB TX 경계	DB Starter	\$4
내부 호출	domainClient.execute()	CPF Domain 간 Typed 호출	secure-api	\$8
외부 연계	@CpfClient / @CpfTimeout / @CpfRetry	외부 호출 정책	secure-api	\$9
Context	CpfContexts.transactionId()/currentExecutionId()	거래·실행 상관관계	cpf-starter	\$12
공통 코드	codeService.required()	필수 공통 코드 조회	cpf-starter/common	\$13
메시지	messageService.resolve()	Locale/Argument 적용 메시지	cpf-starter/common	\$13
파라미터	parameters.requiredValue()	환경/업무 기준값 조회	cpf-starter/common	\$13
영업일	calendar.nextBusinessDay()	영업일 계산	cpf-starter/common	\$13
Logging	@CpfLogging / @CpfPerformance	업무 로그·성능 측정	cpf-starter	\$12
멥등성	@CpfIdempotent	중복 ·IN_PROGRESS-UNKNOWN-Replay	cpf-starter	\$12
권한	@CpfPermission	업무 권한 검사	secure-api	\$12
승인	@CpfApprovalRequired	위험 작업 승인/사유	secure-api	\$12
Audit	@CpfAudit	변경·위험 작업 감사	CPF 운영 계약	\$12
Cache	cacheAside.getOrLoad()	Cache-aside 기본 패턴	Cache Starter	\$10
Messaging	messagePublisher.publish()	Provider-neutral 메시지 발행	Messaging Starter	\$11
Messaging	@CpfMessageListener	표준 Consumer 등록	Messaging Starter	\$11
검증	cpfVerifyFast	개발 중 빠른 계약/정적 Gate	개발 Script	\$2~14

Golden Path 위 Summary 는 일반 업무 개발자가 먼저 익힐 공개 Surface 만 보여줍니다. CpfServiceCaller, Router, Broker Bridge 같은 고급 API 는 해당 장의 Advanced 절 또는 TOP 100 에서 필요할 때 확인합니다.

기본 원칙 업무 Source 는 Public API/Starter 를 사용하고 Provider/internal 구현을 직접 참조하지 않습니다. Generator 로 생성한 업무 Domain 의 Domain Base 가 있는 경우 Framework Base 를 직접 우회하지 않습니다.

2. 실행·테스트·검증 명령

[전체 목차로](#)

실행 위치: 프로젝트 Root | CPF 개발 Script: .\cpf-tools\build\tools\cpf-dev.ps1 | Gradle Wrapper: .\gradlew.bat

2.1 CPF 개발 Script

용도	명령(Action)	주요 옵션	참고사항
전체 통합 실행	run-local	-ResourceProfile local/dev/test/stg/prod	일반 Online 개발
전체 빌드	build	-ResourceProfile	Build + 품질 Gate
전체 Java Test	test	-ResourceProfile	전체 회귀
빠른 검증	verify-fast	-ResourceProfile	계약-Catalog-정적 Gate
전체 로컬 검증	verify-full	-ResourceProfile / -OutputRoot	Java·Frontend·DB·Runtime
Batch 실행	run-batch	-ResourceProfile	Batch 개발
실행 상태/종료	status / stop	-	Local Runtime 확인/종료
모듈/자원 확인	modules / resource	-ResourceProfile	Public Starter / 자원 정책

pwsh .\cpf-tools\build\tools\cpf-dev.ps1 <Action> -ResourceProfile local
예: pwsh .\cpf-tools\build\tools\cpf-dev.ps1 run-local

2.2 특정 Runtime·서비스만 실행

대상	Gradle Task	용도	참고사항
ADM	cpfRunAdm	ADM 만 기동	운영 기능 단독 확인
BZA	cpfRunBza	BZA 만 기동	업무관리 기능 단독 확인
Gateway	cpfRunGateway	Gateway 만 기동	Route/Target 개발
Batch	cpfRunBatch	Batch Runtime 만 기동	Batch 개발
Education	cpfRunEducation	교육/예제 Runtime	CPF 사용 예 확인

.\gradlew.bat <Gradle Task>
예: .\gradlew.bat cpfRunGateway

2.3 Generator 로 생성한 업무 Domain 실행·Test

대상	Gradle 인자	주요 옵션	참고사항
member Online	-p cpf-member :online:bootRun	-PcpfDbVendor=postgresql	Oracle/PostgreSQL/MariaDB 중 선택
member Test	-p cpf-member test	-	member 업무 코드
external Test	-p cpf-external test	-	external 업무 코드

.\gradlew.bat -p cpf-member :online:bootRun -PcpfDbVendor=postgresql

2.4 변경 모듈만 Test

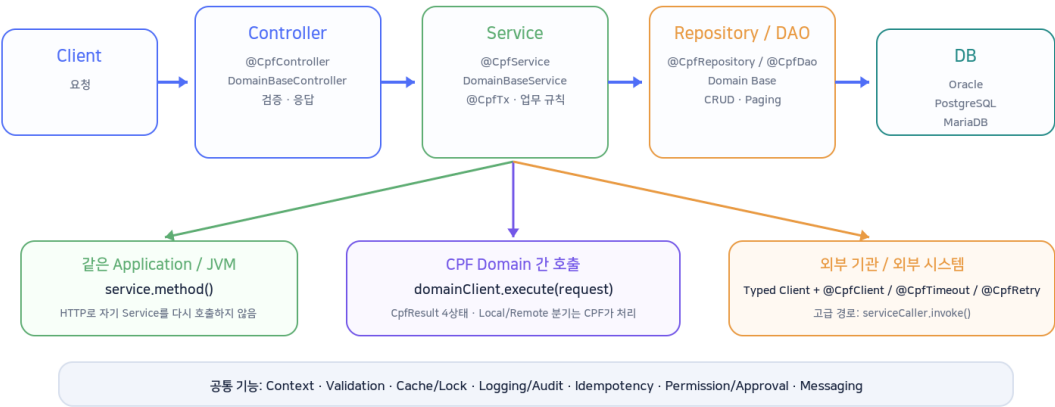
대상	Gradle Task	용도	참고사항
Gateway	:cpf-gateway:test	Gateway 변경 Test	변경 범위 우선 확인
Admin	:cpf-admin:test	ADM Backend 변경 Test	변경 범위 우선 확인
기타 CPF 모듈	:<module>;test	해당 모듈 Test	전체 Test 전 빠른 확인

Runtime 설정 gradle\cpf-runtime\common.properties 와 local/dev/test/stg/prod.properties 에서 환경별 자원 설정을 확인합니다.

3. 업무 API 개발 표준

전체 목차로

업무 API 개발 표준 흐름
기본 업무 계층과 호출 경계를 구분하면 어떤 CPF API를 써야 하는지 빠르게 판단할 수 있습니다.



Controller 는 요청 경계, Service 는 업무 규칙·Transaction·호출 조합, Repository/DAO 는 DB 접근을 담당합니다.

영역	표준 Annotation / Base	자주 쓰는 기능	개발 시 판단
Controller	@CpfController · DomainBaseController	Context/검증/ok/created/noContent	HTTP 경계와 입력·응답 처리
Service	@CpfService · DomainBaseService	requireText · @CpfTx · Domain/외부 호출	업무 규칙과 TX 경계
Repository/DAO	@CpfRepository / @CpfDao · DomainBaseRepository	statementId · boundedSize · executionFacts	CRUD/Paging/DB 접근
내부 호출	Generated Typed Client	execute(request) → CpfResult<T>	Local/Remote 분기 직접 구현 안 함
외부 연계	@CpfClient · @CpfTimeout · @CpfRetry	Typed Client/Adapter 에 호출 정책 적용	일반 업무는 이 패턴을 우선

4. Transaction

전체 목차로

Transaction 표준 패턴

Service method의 @CpfTx가 Local DB Transaction 경계를 만들고, Remote/외부 호출은 같은 원자성으로 가정하지 않습니다.



중요

self-invocation은 Proxy 경계를 확인하고, Remote timeout과 DB Transaction timeout을 같은 값으로 보지 않습니다.

4.1 기본 사용

일반 업무 Transaction 은 Service method 에 `@CpfTx`를 붙입니다. `'id'`, `'name'`, `'ownerDomain'`을 지정하고 나머지는 필요한 경우만 추가합니다.

```
@CpfTx(
    id = "MBR_UPDATE_SAMPLE",
    name = "MBR_SAMPLE_TX_UPDATE",
    ownerDomain = "MBR"
)
public SampleItem update(UpdateCommand command) {
    // validate → repository update → response
}
```

4.2 옵션을 붙이면 어떻게 달라지는가

목적	추가하는 옵션	적용 예	선택 기준	주의
일반 업무	기본값(REQUIRED)	@CpfTx(...)	호출 중 TX 가 있으면 참여	대부분 이 형태
조회 전용	<code>readOnly = true</code>	@CpfTx(..., <code>readOnly=true</code>)	조회 Service	쓰기 동작을 섞지 않음
별도 Commit	<code>propagation = REQUIRES_NEW</code>	@CpfTx(..., <code>propagation=REQUIRES_NEW</code>)	감사/독립 기록 등 정말 분리 필요	남용 시 업무 원자성 분리
격리 수준	<code>isolation = ...</code>	@CpfTx(..., <code>isolation=SERIALIZABLE</code>)	동시성 요구가 명확할 때	DB 특성/부하 확인
DB TX Timeout	<code>timeoutSeconds = 5</code>	@CpfTx(..., <code>timeoutSeconds=5</code>)	DB 작업 상한이 필요할 때	Remote timeout 과 별개
TM 지정	<code>transactionManager = "..."</code>	특정 DataSource 사용	여러 TM 이 실제 구성된 경우	기본 프로젝트는 생략

```
// 조회 전용
@CpfTx(id="MBR_SEARCH", name="MBR_SEARCH_TX", ownerDomain="MBR", readOnly=true)
public List<SampleItem> search(...) { ... }

// 별도 Commit 이 꼭 필요한 기록
@CpfTx(id="MBR_AUDIT", name="MBR_AUDIT_TX", ownerDomain="MBR",
    propagation=Propagation.REQUIRES_NEW)
public void writeAudit(...) { ... }
```

Remote/외부 호출 Local DB Transaction 과 Remote Domain/외부 시스템의 처리 결과를 하나의 원자적 Transaction 으로 가정하지 않습니다. 외부 결과가 UNKNOWN 이면 조회·대사 후 재처리를 판단합니다.

self-invocation 같은 객체 내부에서 Transaction method 를 직접 호출하면 Spring Proxy 경계를 통과하지 않을 수 있습니다. 별도 Bean 경계 또는 호출 구조를 확인합니다.

5. Controller

전체 목차로

Controller 는 HTTP 경계에서 Context·입력 검증·표준 응답을 처리합니다. Generator 로 생성한 업무 Domain 의 `DomainBaseController`를 상속하고 `@CpfController`를 사용합니다.

기능	사용 API / 메소드	용도	주요 옵션 / 반환	참고
CPF Controller 등록	@CpfController	CPF 표준 HTTP Controller	contextRequired=true 기본	일반 업무 기본값
현재 Context	requireCurrentContext()	전체 CpfContext 접근	CpfContext	Context 없으면 오류
Context Snapshot	requireContext()	비민감 Snapshot 사용	CpfContextSnapshot	전파/진단 시
필수 문자열	requireText(value, field)	null/blank 거부 + trim	String	간단 경계 검증
필수 값	requireValue(value, field)	null 거부	T	필수 객체값
업무 조건	requireRule(condition, message)	조건 검증	void	실패 시 표준 검증
200 응답	ok(body)	성공 응답	ResponseEntity<T>	조회/수정 결과
201 응답	created(location, body)	생성 응답	ResponseEntity<T>	등록 API
204 응답	noContent()	본문 없는 성공	ResponseEntity<Void>	삭제 등
상관관계	executionFacts(operation)	transaction/execution/actor/tenant	CpfWebExecutionFacts	로그·감사 보조

```
@CpfController
@RequestMapping("/api/members")
public class MemberController extends MemberBaseController {
    @GetMapping("/{id}")
    public ResponseEntity<MemberDto> detail(@PathVariable String id) {
        String memberId = requireText(id, "id");
        return ok(service.detail(memberId));
    }
}
```

6. Service

전체 목차로

Service 는 업무 규칙의 중심입니다. `@CpfService`를 사용하고 Domain Base 가 제공하는 helper 와 `@CpfTx`, 내부/외부 호출 API 를 조합합니다.

기능	사용 API / 메소드	용도	적용 위치	참고
Service 등록	@CpfService	CPF 업무 Service	Class	contextRequired=true 기본
필수 문자열	requireText(value, field)	입력 정규화/필수값	Service method	CpfBaseService 제공
Transaction ID	requireTransactionId()	현재 거래 식별자	Domain Base	member/external 예제
거래 Sequence	transactionSequence()	동일 거래 내 순서	Domain Base	Context 기반
Actor	actorId()	작업자 식별	Domain Base	없으면 Domain code fallback

기능	사용 API / 메소드	용도	적용 위치	참고
Local DB TX	@CpfTx	업무 DB 원자성	public Service method	\$4
내부 Domain 호출	Typed Client.execute()	다른 Domain 서비스 호출	Service	\$8
외부 연계	@CpfClient + Typed Client	외부 기관/시스템 호출	Service/Adapter	\$9
고급 외부 연계	CpfServiceCaller.invoke()	Registry/Health/Failover 포함 호출	Adapter/Runtime	\$9

Service에 넣을 것 업무 규칙, Transaction 경계, 내부 Domain/외부 연계 호출 조합, 결과 상태 판단을 Service에서 관리합니다. HTTP 세부와 SQL 세부는 각각 Controller/Repository로 분리합니다.

7. DB·Repository/DAO

[전체 목차로](#)

프로젝트가 선택한 Persistence 방식(JDBC/MyBatis/JPA)을 그대로 사용하되 CPF Repository/DAO 경계와 Context·Paging guard를 적용합니다.

기능	CPF API / helper	용도	선택 기준	참고
Repository 등록	@CpfRepository	Repository/Port/Concrete 등록	기본 표준	contextRequired=true
Class DAO 등록	@CpfDao	JDBC/MyBatis DAO	DAO 구조를 쓰는 경우	JPA에 강제하지 않음
Statement ID	statementId(namespace, statement)	MyBatis ID 조합	MyBatis	문자열 검증 포함
Page size 제한	boundedSize(requested, default, max)	무제한 size 방지	Paging	Domain Base에서 정책 고정 가능
Context	requireCurrentContext()/requireContext()	tenant/actor/transaction 접근	필요한 Query/Audit	Provider 직접 참조 금지
진단 정보	executionFacts(operation)	DB 실행 상관관계	로그/진단	비민감 값

7.1 조회 방식 선택

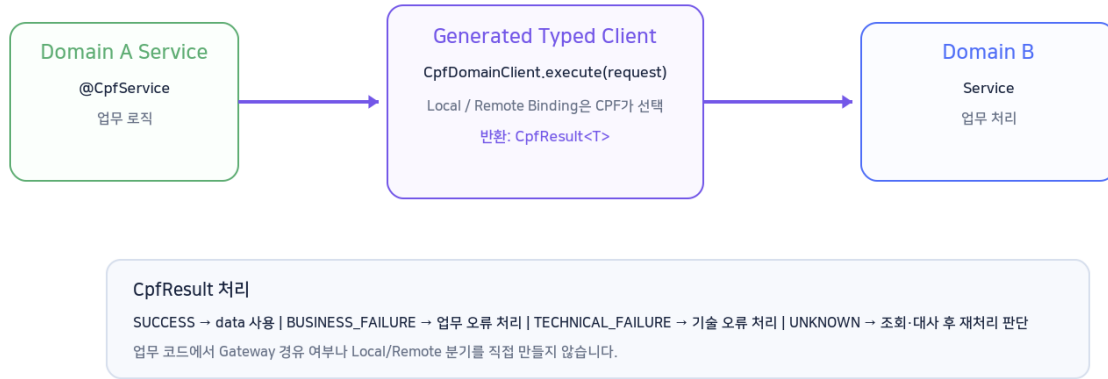
목적	권장 방식	판단 기준	주의
전체 건수도 필요	Page	페이지 번호 + total count 필요	Count Query 비용 고려
다음 페이지 여부만	Slice	total count 불필요	목록 탐색에 적합
대량/연속 탐색	Cursor	정렬 Key가 안정적	동일 정렬/Key 계약 필요
대량 변경	Bulk	한 건 반복보다 Bulk API/SQL	TX/Lock 영향 확인
동시 수정 보호	DB Lock / CPF Lock	DB row 보호 vs 분산 자원 보호 구분	Lock 범위 최소화

8. 내부 Domain 호출

[전체 목차로](#)

내부 Domain 호출 개발 표준

업무 Service는 Generated Typed Client를 주입받아 execute()를 호출하고 CpfResult 상태를 처리합니다.



업무 Service는 대상 Domain의 Generated Typed Client를 주입받아 `execute(request)`를 호출합니다. Local/Remote 배치 위치에 따라 업무 코드에 분기문을 만들지 않습니다.

항목	사용 API	용도	개발자가 할 일	CPF가 처리
Client 계약	CpfDomainClient<I,O>	Typed Domain 호출	Generated Client 주입	공통 호출 계약
호출	execute(request)	Domain 거래 실행	Request 생성·호출	Local/Remote Binding
결과	CpfResult<T>	4 상태 결과	상태별 업무 처리	Recovery 정보 포함
SUCCESS	result.isSuccess() 등	성공 확정	data 사용	상태 표준화
UNKNOWN	result.isUnknown() 등	결과 불명	대사/조회 후 재처리	Recovery 정보

8.1 최소 호출 예제

```
@CpfService
public class OrderService {
    private final ExternalDomainClient externalClient;

    public CpfResult<CpfDomainPingResponse> callExternal(String requestId) {
        return externalClient.execute(new CpfDomainPingRequest(requestId));
    }
}
```

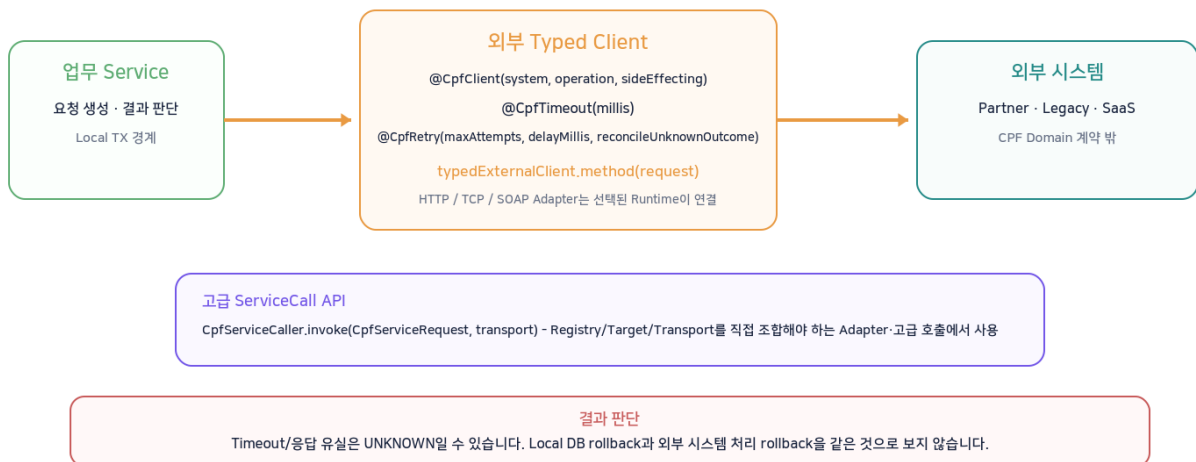
Gateway와 구분 내부 Domain 호출은 Gateway 사용 여부와 별개입니다. 외부 진입 Gateway를 구성해도 Domain 간 업무 호출은 Typed Client 경계를 사용합니다.

9. 외부 연계

[전체 목차로](#)

외부 연계 개발 표준

일반 업무는 CPF 연계 Annotation이 붙은 Typed Client를 우선하고, 고급 Adapter에서만 ServiceCall API를 직접 사용합니다.



외부 기관·Partner·Legacy 처럼 CPF Domain 계약 밖의 대상은 외부 연계로 구분합니다. 일반 업무 코드는 Client/Adapter method에 CPF Integration Annotation을 적용하고 실제 HTTP/SOAP/TCP 호출 구현은 Adapter에 둡니다. Registry·Health·Failover까지 직접 조합해야 하는 고급 Runtime/Adapter에서는 `CpfServiceCaller.invoke()`를 사용합니다.

9.1 일반 업무 Client/Adapter 패턴

기능	Annotation / API	용도	주요 옵션	선택 기준
대상 등록	@CpfClient	외부 대상/거래를 CPF 정책에 등록	system / operation / sideEffecting	외부 호출 method/type
Timeout	@CpfTimeout	전체 호출 시간 상한	millis	대상 SLA 기준
Retry	@CpfRetry	일시 기술 실패 재시도	maxAttempts / delayMillis	역등 호출 중심
UNKNOWN 대사	reconcileUnknownOutcome	결과불명 시 대사 요구	true/false	Side Effect 거래 중심

```
// 교육용 업무 Adapter 예제 - Annotation은 실제 CPF Public API
@CpfClient(system = "BANK-HOST", operation = "ACCOUNT-INQUIRY", sideEffecting = false)
@CpfTimeout(millis = 3000)
@CpfRetry(maxAttempts = 3, delayMillis = 200)
public BankResponse inquire(BankRequest request) {
    return transport.inquire(request); // HTTP/SOAP/TCP 구현은 Adapter 책임
}

@CpfClient(system = "BANK-HOST", operation = "TRANSFER", sideEffecting = true)
@CpfTimeout(millis = 5000)
@CpfRetry(maxAttempts = 2, delayMillis = 300, reconcileUnknownOutcome = true)
public TransferResponse transfer(TransferRequest request) {
    return transport.transfer(request);
}
```

Transaction 경계 Local DB `@CpfTx`와 외부 기관 처리는 하나의 원자적 Transaction이 아닙니다. Timeout/응답 유실이면 상태 처리가 완료됐을 수 있으므로 Side Effect 거래는 UNKNOWN과 대사 경로를 반드시 고려합니다.

9.2 고급 Service Call API

대상 Registry, Health, Retry/Failover/Circuit과 Transport를 직접 조합하는 Runtime/Adapter에서는 `CpfServiceCaller.invoke()`를 사용합니다. 일반 업무에서 단순히 외부 HTTP를 호출하려고 이 저수준 API부터 사용할 필요는 없습니다.

기능	사용 API / 필드	용도	주요 선택	참고
호출	CpfServiceCaller.invoke(request, transport)	표준 Service Call 실행	Transport 구현/Adapter	Registry/Health/Retry 정책 적용
대상	serviceId / endpointCode	Service/Endpoint 식별	serviceId 필수	instanceId 는 필요 시
경로	httpMethod / requestPath	HTTP 계열 요청 정보	기본 GET //	Transport 별 의미 확인
Timeout	timeoutMillis	연계 응답 상한	대상 SLA 기준	DB TX timeout 과 별도
Retry	retryCount	일시 실패 재시도	먹등 요청 중심	Side Effect 주의
결과	success()/businessFailure()/technicalFailure()/unknown()	4 상태 판정	helper 사용	문자열 status 직접 비교 지양
Recovery	recoveryId/recoveryAction	UNKNOWN 후 대사	recoveryRequired()	재처리 전 확인

```
CpfServiceRequest request = CpfServiceRequest.builder("partner-service")
    .endpointCode("member-query")
    .httpMethod("GET")
    .requestPath("/members/" + memberId)
    .timeoutMillis(1500)
    .retryCount(1)
    .build();
```

```
CpfServiceResult<MemberResponse> result = caller.invoke(request, transport);
if (result.success()) return result.responseBody();
if (result.unknown()) { /* 조화·대사 후 판단 */ }
throw mapFailure(result);
```

10. Cache·Lock

전체 목차로

Cache 는 Provider 구현이 아니라 `CpfCachePort`와 `CpfCacheAsideService`를 사용합니다. 단순 조회/저장과 Cache-aside 를 구분해 선택합니다.

목적	API / 메소드	용도	주요 옵션	선택 기준
조회	CpfCachePort.get(key)	직접 Cache 조회	Key namespace/tenant	Cache miss 직접 처리
저장	put(key,value,ttl)	TTL 포함 저장	ttl	명시적 갱신
단건 삭제	evict(key)	Key 무효화	-	변경 후 특정 값 제거
영역 삭제	evictNamespace(tenant, namespace)	Namespace 전체 무효화	tenant/namespace	대량 기준정보 갱신
조회+원본 Load	CpfCacheAsideService.getOrLoad(...)	miss 시 Loader 실행 + single-flight	ttl/negativeTtl/cacheNull/failOpen	읽기 업무 기본 패턴
분산 Lock	CpfDistributedLockPort	동일 자원 중복 처리 방지	wait/lease	DB row lock 과 목적 구분

failOpen Cache 장애를 원본 조회로 우회할 수 있지만 원본 Loader 실패를 Cache miss 로 숨기지 않습니다. 업무 중요도에 따라 failOpen 을 선택합니다.

11. Messaging

전체 목차로

업무 발행은 Provider 별 Kafka/JMS API 가 아니라 공통 Publisher 를 사용하고, 수신 method 에는 `@CpfMessageListener`를 적용합니다.

기능	공개 API / Annotation	용도	주요 옵션	참고
기본 발행	CmnMessagePublisher.publish(key,payload)	기본 목적지 발행	key/payload	Broker 구현 비의존
목적지 발행	publish(destination,key,payload,headers)	목적지/전파 Header 지정	destination/headers	Context Header 최소화
수신	@CpfMessageListener	Consumer method 등록	destination/consumerGroup	Broker Annotation 직접 의존 줄임
먹등	idempotencyRequired=true	중복 수신 보호	기본 true	Side Effect Consumer 필수 검토
Context	contextRequired=true	CPF Context 요구	기본 true	관리 메시지 기본

12. Logging·Audit·Security

[전체 목차로](#)

업무 로그, 감사, 권한은 목적이 다릅니다. 일반 로그에 민감정보를 넣지 않고 위험 동작은 권한·승인·감사를 함께 적용합니다.

목적	Annotation / API	주요 옵션	적용 기준	주의
업무 실행 로그	@CpfLogging	operation/mode/ includeArguments/allowlist	업무 method 요약	Argument/Result 기본 미 포함
변경 감사	@CpfAudit	action/reasonRequired/ includeSafeResultSummary	변경·위험 업무	일반 로그와 별도
권한	@CpfPermission	value[] / all	업무 권한 검사	fail-closed
승인	@CpfApprovalRequired	action/approvals/ reasonRequired	위험 조치 전	approvalId/reason 전달
Context	CpfContext / executionFacts	transactionId/executionId/ actorId/tenantId	상관관계	민감정보 직접 기록 금지

13. 공통 Code·Parameter·Message·Calendar

[전체 목차로](#)

기준정보와 메시지/영업일은 저장소를 직접 조회하지 않고 CPF Common Service 를 주입받아 사용합니다.

기능	공개 메소드	용도	선택 기준	예
공통코드	values(group)	그룹 전체 조회	Select 목록/검증 기준	STATUS
공통코드	find()/required()	코드 단건 조회	없어도 됨 / 반드시 있어야 함	STATUS/ACTIVE
파라미터	requiredValue(key)	문자열 필수값	문자열 설정	MAX_PAGE_SIZE
파라미터	findValue()/ requiredValue(key,type)	Typed 값 조회	Integer/Boolean/Duration/ Date 등	TIMEOUT
메시지	resolve(code,locale,args)	Locale/Argument 적용 메시지	사용자 메시지	MBR001
영업일	isBusinessDay()	영업일 여부	Calendar 기준	BANK
영업일	shiftBusinessDay()/next/ previous	영업일 이동	D+N 계산	정산일

14. Test·오류 확인

[전체 목차로](#)

변경한 업무 단위의 Test 를 먼저 실행하고, 정상·검증 오류·업무 오류·기술 오류·UNKNOWN 경계를 각각 확인합니다.

대상	우선 Test	확인할 것	명령 예
Controller	Web/MVC Test	검증·Context·HTTP 상태·오류 응답	.\gradlew.bat -p cpf-member :online:test
Service	Unit/Spring Test	업무 규칙·@CpfTx 경계·호출 결과	동일
Repository	DB Integration Test	CRUD/Paging/Lock/rollback	DB Vendor 설정 포함
Domain 호출	Client/Router Test	SUCCESS/실패/UNKNOWN	Typed Client Consumer 포함
외부 연계	Transport/ServiceCall Test	Timeout/Retry/UNKNOWN/Recovery	Mock/Fake 대상
전체 변경	cpfVerifyFast → 전체 Test	계약·Catalog·정적 Gate 후 회귀	cpf-dev.ps1 verify-fast / test

오류 확인 표준 오류 응답의 transactionId/executionId 를 보존해 로그·Trace·ADM 에서 같은 실행을 찾을 수 있게 합니다.

15. Starter 선택·Reference

[전체 목차로](#)

새 기능이 필요하면 먼저 Public Profile/Starter 범위를 확인하고 Provider 를 선택합니다. 업무 코드에서 Internal Starter 를 직접 추가하지 않습니다.

개발 목적	Public 선택	Provider/방식	업무 코드에서 보는 것
Web API	cpf-starter-web-api / secure-api	일반 / 인증 API	@CpfController · Security API
DB	cpf-starter-data-jdbc/mybatis/jpa	JDBC / MyBatis / JPA	Repository/DAO
Cache	cpf-starter-cache-caffeine/redis/valkey	Local / Shared	CpfCachePort
Messaging	cpf-starter-messaging-kafka/rabbitmq/jms/ibm-mq	Kafka / RabbitMQ / JMS / IBM MQ	Publisher/Listener
Session	cpf-starter-session-jdbc/valkey	JDBC / Valkey	Session 기능
기타 선택 기능	object-storage-s3 / graphql / realtime	S3-compatible / GraphQL / SSE	각 Public 기능 API

15.1 CPF 공통 API 와 Native 사용 경계

구분	대표 영역	개발 기준
CPF 표준 우선	TX · Domain/외부 호출 · Context · Logging/Audit · 권한 · 먹등 · Messaging	Public API/Annotation 우선
Native 허용	Collection · String/Date · 순수 변환 · 일반 Bean wiring	CPF 계약과 무관한 코드
직접 사용 지양	Domain WebClient · Provider API · 자체 Retry/Idempotency · Internal Starter	Public/Adapter 경계 사용

16. CPF 개발 기능 TOP 100

[전체 목차로](#)

업무 개발에서 자주 찾는 CPF 공개 기능과 실행 명령을 10 개 기능군으로 정리했습니다. 각 표에서는 기능의 존재와 선택 기준만 빠르게 확인하고, 실제 적용은 상세 장의 표준 흐름과 예제를 사용합니다.

사용 기준 Golden 은 일반 업무 개발자가 우선 익힐 항목, Capability 는 해당 기능을 사용할 때 확인할 항목, Advanced 는 Adapter/Framework 성격 코드에서 주로 사용하는 계약입니다. TOP 100 은 API 개수를 채우기 위한 목록이 아니라 실제 선택용 Quick Reference 입니다.

A. 업무 개발 기본

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
001	Golden	@CpfController	CPF Web Controller 등록	cpf-starter-web-api	contextRequired	\$5
002	Golden	@CpfService	CPF Business Service 등록	cpf-starter	contextRequired	\$6
003	Golden	@CpfRepository	Repository 표준 등록	DB Starter	contextRequired	\$7
004	Capability	@CpfDao	JDBC/MyBatis DAO 등록	DB Starter	contextRequired	\$7
005	Capability	@CpfDto	DTO 계약/검증/민감정보 metadata	cpf-starter	contractVersion / validationRequired / sensitive	\$5
006	Capability	requireText()	필수 문자열 검증	cpf-starter / web-api	fieldName	\$5~7
007	Capability	requireValue()	필수 객체/값 검증	web-api / DB Starter	fieldName	\$5~7
008	Capability	requireRule()	업무 조건 검증	web-api / DB Starter	condition / message	\$5~7
009	Capability	ok() / created()	200/201 표준 응답	cpf-starter-web-api	body / location	\$5
010	Capability	noContent()	204 표준 응답	cpf-starter-web-api	-	\$5

B. Context·Validation·Paging

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
011	Capability	CpfContexts.requireCurrent()	현재 CPF Context 필수 조회	cpf-starter	Context 없으면 fail	\$5~12
012	Capability	CpfContexts.requireSnapshot()	전파 가능한 Context Snapshot	cpf-starter	비동기/진단	\$12
013	Golden	CpfContexts.transactionId()	현재 Transaction ID	cpf-starter	필수 Context	\$12
014	Golden	CpfContexts.currentExecutionId()	현재 Execution ID	cpf-starter	Context 없으면 null	\$12
015	Capability	CpfContexts.currentSegmentId()	현재 Segment ID	cpf-starter	Context 없으면 null	\$12
016	Capability	CpfContexts.capture()/run()	비동기 Context 전달	cpf-starter	Snapshot 캡처 후 실행	\$12
017	Capability	CpfValidation.requireNotEmpty()	Collection 비어 있음 검증	cpf-starter	name	\$5~6
018	Capability	CpfPages.request()	Page 요청 정규화	cpf-starter	기본 20 / 최대 500	\$7
019	Capability	CpfPage.hasNext()/totalPages()	Page 응답 탐색	cpf-starter	total count 필요 시	\$7
020	Capability	CpfSlice / CpfCursorPage	경량/대량 연속 조회	DB Starter	Slice 또는 Cursor 선택	\$7

C. Transaction·Persistence

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
021	Golden	@CpfTx	CPF Local DB Transaction	DB Starter	id / name / ownerDomain	\$4
022	Capability	propagation=REQUIRED	기존 TX 참여 또는 신규 시작	DB Starter	기본값	\$4
023	Capability	propagation=REQUIRES_NEW	독립 Local TX	DB Starter	별도 Commit 필요 시	\$4
024	Capability	readOnly=true	조회 전용 TX	DB Starter	쓰기 섞지 않음	\$4
025	Capability	timeoutSeconds	DB TX 시간 상한	DB Starter	Remote timeout 과 별도	\$4
026	Capability	isolation	DB 격리수준 선택	DB Starter	DB 특성/부하 검토	\$4
027	Capability	transactionManager	특정 TransactionManager 지정	DB Starter	다중 TM 구성 시	\$4
028	Capability	statementId()	MyBatis Statement ID 안전 조합	cpf-starter-data-mybatis	namespace / statement	\$7
029	Capability	boundedSize()	Paging size 상한 적용	DB Starter	default / max	\$7

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
030	Capability	CpfSearchRepositoryPort.page/slice/cursor	검색 Paging 방식 선택	DB Starter	Page / Slice / Cursor	\$7

D. 내부 Domain 호출·Result

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
031	Golden	CpfDomainClient.execute()	CPF Domain 간 Typed 호출	cpf-starter-secure-api	AUTO/LOCAL/REMOTE 는 Runtime	\$8
032	Capability	CpfResult.isSuccess()	성공 확정 판정	secure-api	SUCCESS	\$8
033	Capability	CpfResult.isBusinessFailure()	업무 실패 판정	secure-api	재시도 대상 아님	\$8
034	Capability	CpfResult.isTechnicalFailure()	기술 실패 판정	secure-api	Retry 정책 별도	\$8
035	Capability	CpfResult.isUnknown()	결과불명 판정	secure-api	Blind retry 금지	\$8
036	Capability	CpfResult.requireData()	SUCCESS 데이터 추출	secure-api	비성공이면 예외	\$8
037	Capability	CpfResult.fold(4-way)	4 상태별 후처리	secure-api	성공/업무/기술/UNKNOWN	\$8
038	Capability	CpfResult.fold(3-way)	실패 통합 + UNKNOWN 분리	secure-api	단순 연동 업무	\$8
039	Capability	CpfResult.success()/businessFailure()	Domain 결과 생성	secure-api	Adapter/Operation 구현	\$8
040	Capability	CpfResult.technicalFailure()/unknown()	기술/결과불명 결과 생성	secure-api	UNKNOWN 은 recoveryInfo 필수	\$8

E. 외부 연계

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
041	Golden	@CpfClient	외부 호출을 CPF Integration 정책에 등록	cpf-starter-secure-api	system / operation / sideEffecting	\$9
042	Golden	@CpfTimeout	외부 호출 Timeout 선언	cpf-starter-secure-api	millis	\$9
043	Golden	@CpfRetry	외부 호출 Retry 정책	cpf-starter-secure-api	maxAttempts / delayMillis	\$9
044	Capability	reconcileUnknownOutcome	UNKNOWN 대사 요구 표시	cpf-starter-secure-api	쓰기 거래 중심	\$9
045	Advanced	CpfServiceCaller.invoke()	Registry/Health/Retry 포함 고급 호출	cpf-starter-secure-api	request / transport	\$9
046	Capability	CpfServiceRequest.builder()	Service Call 요청 생성	cpf-starter-secure-api	serviceId 필수	\$9
047	Capability	endpointCode()/requestPath()	대상 Endpoint/경로 지정	cpf-starter-secure-api	endpointCode / path	\$9
048	Capability	timeoutMillis()/retryCount()	호출별 Timeout/Retry	cpf-starter-secure-api	대상 SLA/먹등성	\$9
049	Capability	CpfServiceResult.success()/businessFailure()	외부 결과 판정	cpf-starter-secure-api	문자열 status 비교 지양	\$9
050	Capability	technicalFailure()/unknown()/recoveryRequired()	기술/UNKNOWN/복구 필요 판정	cpf-starter-secure-api	UNKNOWN 후 대사	\$9

F. 공통 업무 기능

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
051	Capability	codeService.values()	코드 그룹 전체 조회	cpf-starter / common	group	\$13
052	Capability	codeService.find()	코드 선택 조회	cpf-starter / common	group / value	\$13
053	Golden	codeService.required()	필수 코드 조회	cpf-starter / common	없으면 명확히 실패	\$13
054	Golden	messageService.resolve()	공통 메시지 해석	cpf-starter / common	code / locale / args	\$13
055	Golden	parameters.requiredValue()	필수 문자열 Parameter	cpf-starter / common	key	\$13
056	Capability	requiredValue(key,type)	Typed Parameter 조회	cpf-starter / common	Integer/Boolean/Duration 등	\$13

No	구분	기능 / API	응도	필요 Starter / 경로	주요 옵션·선택	상세
057	Capability	parameters.findValue()	선택 Typed Parameter	cpf-starter / common	Optional 반환	\$13
058	Capability	calendar.isBusinessDay()	영업일 여부	cpf-starter / common	calendarId / date	\$13
059	Capability	calendar.next/previous/ shiftBusinessDay()	영업일 이동	cpf-starter / common	offset	\$13
060	Capability	templates.render()	활성 Template 렌더	cpf-starter / common	templateCode / channel / variables	\$13

G. Cache·Lock

No	구분	기능 / API	응도	필요 Starter / 경로	주요 옵션·선택	상세
061	Capability	cache.get()	Cache 조회	Cache Provider Starter	CpfCacheKey	\$10
062	Capability	cache.put()	TTL 포함 Cache 저장	Cache Provider Starter	value / ttl	\$10
063	Capability	cache.evict()	단건 무효화	Cache Provider Starter	key	\$10
064	Capability	cache.evictNamespace()	Namespace 무효화	Cache Provider Starter	tenant / namespace	\$10
065	Capability	cache.health()	Cache 연결/준비 상태	Cache Provider Starter	Provider 상태	\$10
066	Capability	cache.metrics()	Cache hit/miss 등 지표	Cache Provider Starter	운영 진단	\$10
067	Golden	cacheAside.getOrLoad()	Miss 시 원본 Load + single-flight	Cache Provider Starter	CpfCacheOptions	\$10
068	Capability	CpfCacheOptions	Cache-aside 정책	Cache Provider Starter	ttl / negativeTtl / cacheNull / failOpen	\$10
069	Advanced	locks.tryAcquire()	분산 Lock 획득	cpf-starter-lock-valkey	wait / lease	\$10
070	Advanced	locks.release()	분산 Lock 해제	cpf-starter-lock-valkey	CpfLockToken	\$10

H. Messaging·Event

No	구분	기능 / API	응도	필요 Starter / 경로	주요 옵션·선택	상세
071	Golden	CmnMessagePublisher.publish(key, payload)	기본 목적지 발행	Messaging Provider Starter	key / payload	\$11
072	Capability	publish(destination,key,payload,he aders)	목적지/전파 Header 발 행	Messaging Provider Starter	destination / headers	\$11
073	Golden	@CpfMessageListener	CPF Message Consumer 등록	Messaging Provider Starter	destination / consumerGroup	\$11
074	Capability	idempotencyRequired	중복 수신 보호	Messaging Provider Starter	기본 true	\$11
075	Capability	contextRequired	Message Context 요구	Messaging Provider Starter	기본 true	\$11
076	Advanced	CpfBrokerBridgePort.publish()	Provider-neutral 저수 준 발행	Messaging Provider Starter	destination / headers	\$11
077	Advanced	CpfBrokerBridgePort.subscribe()	Provider-neutral 구독	Messaging Provider Starter	handler	\$11
078	Advanced	CpfBrokerBridgePort.findRecent()	최근 메시지 조회	Messaging Provider Starter	destination / limit	\$11
079	Advanced	CpfBrokerClient.enqueue()	신뢰성 Queue/Outbox 저수준 발행	Messaging Provider Starter	CpfBrokerPublishRequest	\$11
080	Capability	unknownPort.probe()/reconcile()	Broker UNKNOWN 조 회·대사	Messaging Provider Starter	operatorId / reason	\$11

I. Logging·보안·먹등성

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
081	Golden	@CpfLogging	Context 연계 업무 로그	cpf-starter	operation / mode	\$12
082	Capability	includeArguments + allowlist	허용된 입력만 기록	cpf-starter	기본 false	\$12
083	Capability	includeResult + resultAllowlist	허용된 결과만 기록	cpf-starter	기본 false	\$12
084	Golden	@CpfPerformance	성능 계측	cpf-starter	slowThresholdMillis	\$12
085	Golden	@CpfIdempotent	중복 요청/UNKNOWN/Replay 보호	cpf-starter	ttl / inProgressTimeout / replayResult	\$12
086	Golden	@CpfPermission	업무 권한 검사	cpf-starter-secure-api	value[] / all	\$12
087	Capability	all=true/false	모든 권한/하나 이상 선택	cpf-starter-secure-api	권한 조합	\$12
088	Golden	@CpfApprovalRequired	위험 동작 승인 요구	cpf-starter-secure-api	action / approvals / reasonRequired	\$12
089	Capability	@CpfAudit	변경/위험 작업 Audit	CPF 운영 계약	action / reasonRequired	\$12
090	Capability	executionFacts()	거래/실행/사용자/tenant 진단 정보	web-api / DB Starter	operation	\$5·7·12

J. 개발 실행·검증

No	구분	기능 / API	용도	필요 Starter / 경로	주요 옵션·선택	상세
091	Golden	cpf-dev.ps1 run-local	전체 로컬 통합 Runtime	개발 Script	-ResourceProfile	\$2
092	Capability	cpfRunAdm	ADM 단독 실행	Root Gradle Task	-	\$2
093	Capability	cpfRunBza	BZA 단독 실행	Root Gradle Task	-	\$2
094	Capability	cpfRunGateway	Gateway 단독 실행	Root Gradle Task	-	\$2
095	Capability	cpfRunBatch	Batch Runtime 단독 실행	Root Gradle Task	-	\$2
096	Capability	cpfRunEducation	Education 예제 Runtime	Root Gradle Task	-	\$2
097	Capability	cpf-dev.ps1 build	전체 Build + 품질 Gate	개발 Script	-ResourceProfile	\$2
098	Capability	cpf-dev.ps1 test	전체 Java Test	개발 Script	-ResourceProfile	\$2
099	Golden	cpf-dev.ps1 verify-fast	계약/Catalog/정적 빠른 검증	개발 Script	-ResourceProfile	\$2
100	Capability	cpf-dev.ps1 verify-full	Java/Frontend/DB/Runtime 최대 검증	개발 Script	-ResourceProfile / -OutputRoot	\$2